

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Towards Dev · [Follow publication](#)

Recursion Optimizations in C++ for Low-Latency Software

Techniques for eliminating recursive overhead in performance-critical C++.

10 min read · Apr 8, 2026



Sagar

Following ▾



Listen



Share

... More

The thumbnail image is a slide from a presentation titled "LOW-LATENCY C++ SERIES". The main title is "Recursion Optimizations" in large white font, with the subtitle "in C++ for Low-Latency Software" below it. The slide features several optimization techniques listed in blue rounded rectangles: "Tail Call Elimination", "Explicit Stack", "Loop Rewrite", "Higher Base Level", "Level Collapsing", and "Stack Overflow Safe". A text box explains: "Every recursive call is a stack frame. Every stack frame is wasted cycles. Stop going deeper. Start going faster." Below this are three statistics: "97% fewer function calls with higher base level", "0 stack frames with explicit stack approach", and "8x faster tree traversal iterative vs recursive". On the right, a "CALL STACK" diagram shows five frames (fn(n) to fn(n-4)) with a red box at the bottom stating "Stack overflow risk on deep input". The footer includes "C++17 | x86-64 | Ubuntu 24.04 | GCC 13" and "Low-Latency Series".

Why Recursion is the Enemy for Low-Latency Software?

Alright, movie time again. Remember Groundhog Day? Bill Murray wakes up, same day, same song on the radio, same conversation with Ned Ryerson. Over and over and over. That's recursion. The same function preamble, the same register saves, the

same stack frame allocation — repeating endlessly, accumulating overhead like existential dread.

The iterative version? That's Bill Murray finally learning piano, speaking French, ice-sculpting. Same outcome, no repeated suffering. He figured out how to get the result without reliving the setup a thousand times.

Every recursive call means:

- **Stack frame allocation** — push arguments, return address, saved registers (~20–40 bytes per frame)
- **Instruction cache pollution** — the same function preamble/epilogue executes thousands of times
- **Branch predictor confusion** — the return address changes every frame
- **Stack overflow risk** — default stack is 8MB on Linux; deep recursion eats it fast

On a hot path, recursion is almost never justified. Let's fix that.

Recursion trades performance for elegance.

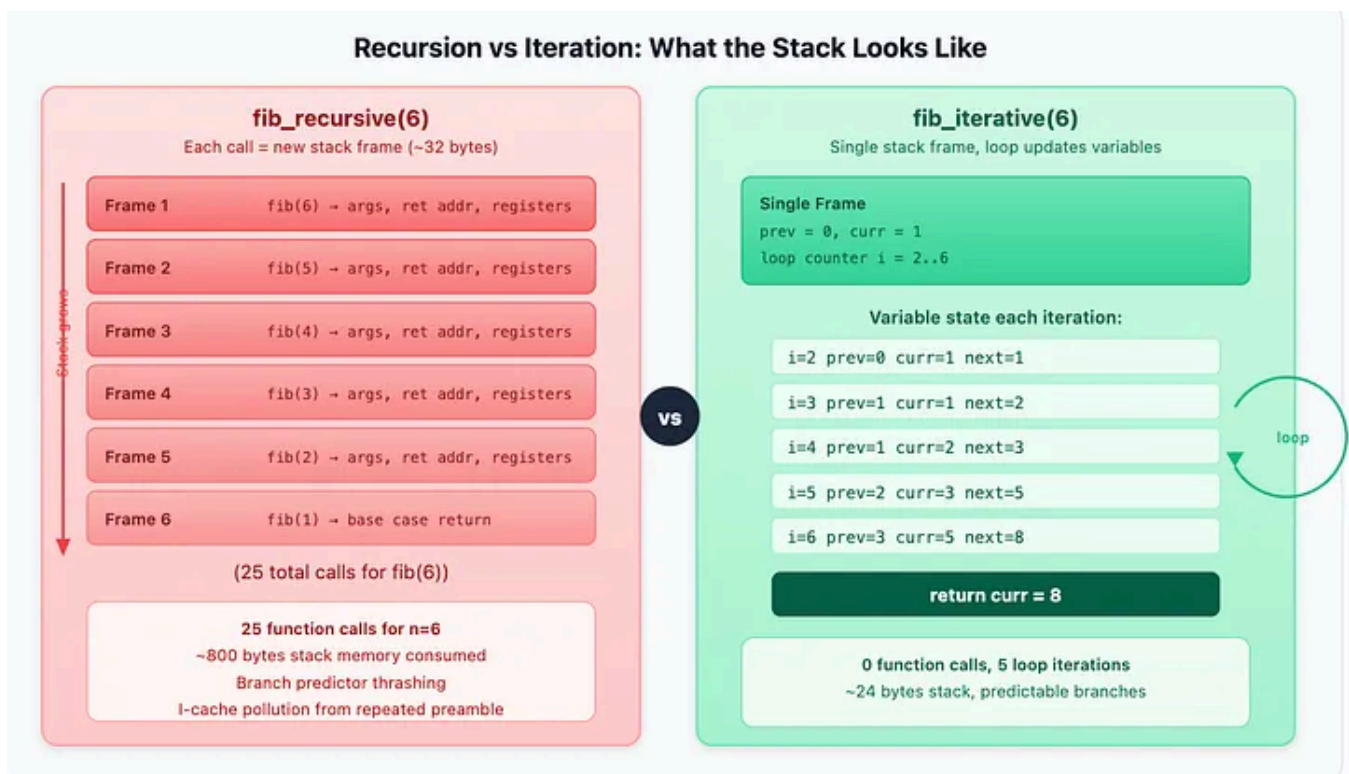
Low-latency systems don't pay for elegance.

1. Avoid Recursion (Completely)

I mean it. If you're writing recursion on a hot path in 2026, you need a very good reason, and "it looks elegant" is not one. We're not in college anymore. Nobody is grading you on how closely your code resembles the mathematical definition.

```
// The canonical "please don't do this in production"
// Exponential time. ~2^n function calls. I have seen this. In production. In f
uint64_t fib_recursive(int n) {
    if (n <= 1) return n;
    return fib_recursive(n - 1) + fib_recursive(n - 2);
}
```

```
// Iterative – O(n) time, O(1) space, zero function call overhead
uint64_t fib_iterative(int n) noexcept {
    if (n <= 1) return n;
    uint64_t prev = 0, curr = 1;
    for (int i = 2; i <= n; ++i) {
        uint64_t next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}
```



fib_recursive(40) makes approximately 2.4 billion function calls.

fib_iterative(40) executes 39 additions and 39 assignments. One of these finishes in nanoseconds. The other might not finish before your patience does.

And before someone says “**but memoization fixes the exponential part**” — yes, it does. You still have 40 function calls with stack frames. The iterative version has zero. On a hot path, zero wins.

If recursion depth is unbounded or runs inside a loop → it does not belong on your hot path.

2. Replace Simple Recursion with a Loop

Any recursion that makes a **single recursive call** per invocation is trivially convertible to a loop. There's no algorithmic reason to use recursion here. It's a style choice, and in low-latency code, style doesn't get to cost cycles.

Linked List Traversal:

```
// Recursive – new stack frame for every node. A 10,000-node list = 10,000 frames
int count_recursive(const Node* node) {
    if (!node) return 0;
    return 1 + count_recursive(node->next);
}
```

```
// Iterative – one stack frame total. Runs on a toaster.
int count_iterative(const Node* node) noexcept {
    int count = 0;
    while (node) {
        ++count;
        node = node->next;
    }
    return count;
}
```

Binary Search:

This one personally offends me when I see it recursive:

```
// Recursive binary search – adds a function call to an O(log n) algorithm
// that's already only ~20 iterations for a million elements. WHY.
int bsearch_recursive(const int* data, int lo, int hi, int target) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (data[mid] == target) return mid;
    if (data[mid] < target)
        return bsearch_recursive(data, mid + 1, hi, target);
}
```

```
    return bsearch_recursive(data, lo, mid - 1, target);  
}
```

```
// Iterative – identical logic, no stack frames, and tighter branch behavior  
int bsearch_iterative(const int* data, int n, int target) noexcept {  
    int lo = 0, hi = n - 1;  
    while (lo <= hi) {  
        int mid = lo + (hi - lo) / 2;  
        if (data[mid] == target) return mid;  
        if (data[mid] < target) lo = mid + 1;  
        else hi = mid - 1;  
    }  
    return -1;  
}
```

GCD — Another Common Offender:

```
// Recursive Euclidean GCD  
int gcd_recursive(int a, int b) {  
    if (b == 0) return a;  
    return gcd_recursive(b, a % b);  
}
```

```
// Iterative – two lines of actual logic  
int gcd_iterative(int a, int b) noexcept {  
    while (b != 0) {  
        int t = b;  
        b = a % b;  
        a = t;  
    }  
    return a;  
}
```

The pattern is always the same:

1. The base case becomes the `while` condition (inverted).
2. The recursive parameters become local variables updated each iteration.

3. The return value is the final state of those variables.

If your recursion has one recursive call, convert it. Today.

3. Replace Complex Recursion with an Explicit Stack

When recursion **branches** — tree traversal, graph DFS, backtracking — you can't use a simple loop. But you absolutely *can* replace the implicit call stack with an explicit one. Same algorithm, same result, zero function call overhead.

Tree Traversal:

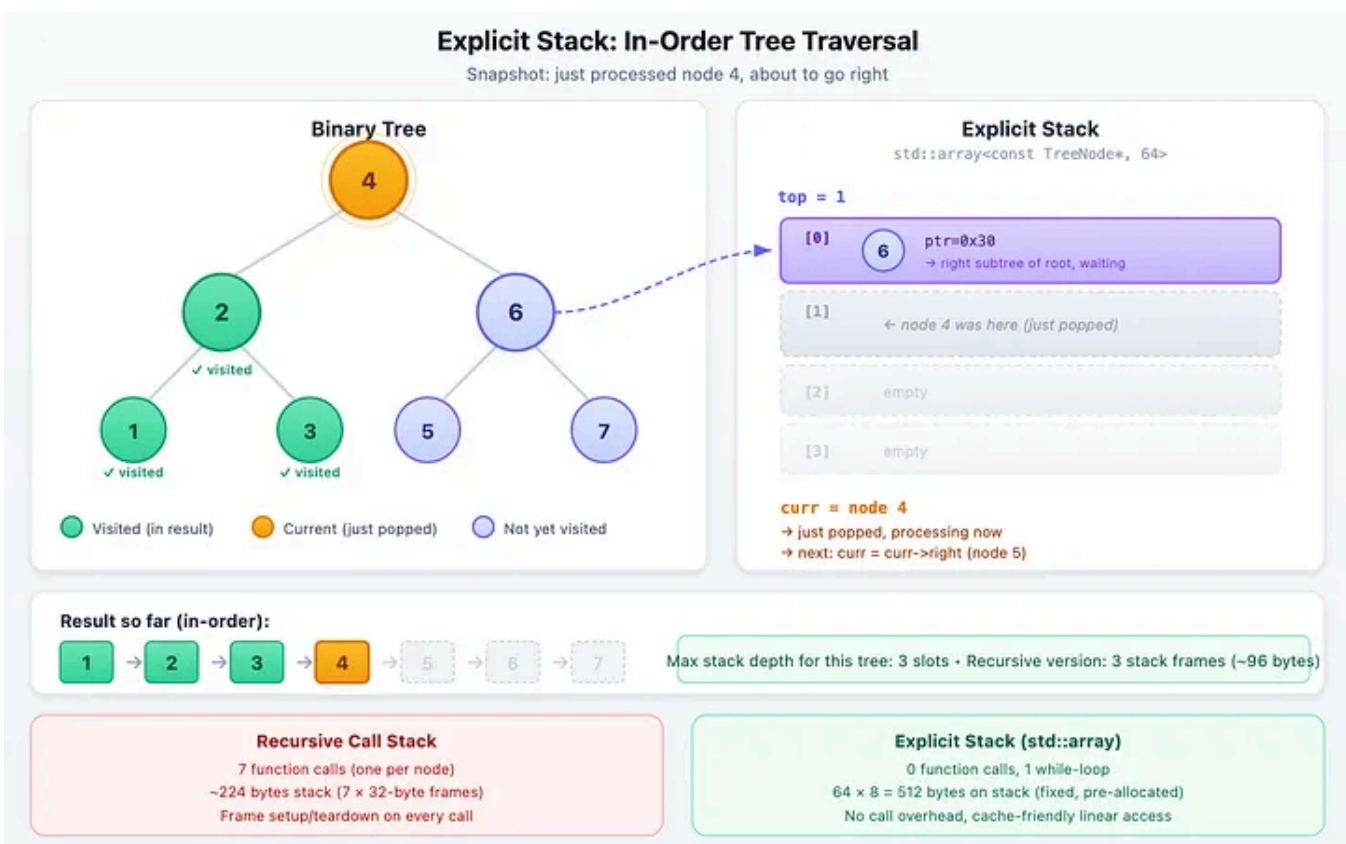
```
// Recursive in-order — one stack frame per node
void traverse_recursive(const TreeNode* node,
                      std::vector<int>& result) {
    if (!node) return;
    traverse_recursive(node->left, result);
    result.push_back(node->value);
    traverse_recursive(node->right, result);
}
```

```
// Explicit stack — same traversal order, no recursion
void traverse_iterative(const TreeNode* root,
                      std::vector<int>& result) {
    // Balanced tree of 1M nodes → max depth ~20.
    // 64 entries is generous. Use std::array to avoid heap allocation.
    std::array<const TreeNode*, 64> stack;
    int top = 0;
    const TreeNode* curr = root;
    while (curr || top > 0) {
        // Dive left
        while (curr) {
            stack[top++] = curr;
            curr = curr->left;
        }
        // Process
        curr = stack[--top];
        result.push_back(curr->value);
        // Go right
        curr = curr->right;
    }
}
```

```
}
}
```

Why this is faster:

ASPECT	RECURSIVE	EXPLICIT STACK
Frame cost per node	~30-40 bytes + call/ret	~8 bytes (one pointer)
Memory layout	Scattered on call stack	Contiguous <code>std::array`</code> in L1
Function call overhead	Full (save/restore registers)	None
Stack overflow risk	Real (8MB default)	You control the limit
Cache behavior	Cold (each frame at different address)	Hot (array stays in L1)



Graph DFS: Same idea, different data structure

```

// Explicit DFS – works on any graph, no recursion
void dfs_iterative(const Graph& g, int start,
                  std::vector<bool>& visited) {
    std::vector<int> stack;
    stack.reserve(g.node_count()); // one allocation up front
    stack.push_back(start);
    while (!stack.empty()) {
        int node = stack.back();
        stack.pop_back();
        if (visited[node]) continue;
        visited[node] = true;
        // Push neighbors – they'll be processed LIFO, same as recursion
        for (int neighbor : g.adjacent(node)) {
            if (!visited[neighbor])
                stack.push_back(neighbor);
        }
    }
}

```

Backtracking / Combinatorial Search

Even complex backtracking algorithms convert cleanly. The “state” you’d normally keep in local variables across recursive calls gets pushed onto your explicit stack as a struct:

```

struct SearchState {
    int position;
    int choices_remaining;
    uint32_t decisions; // bitmask of choices made so far
};

void solve_iterative(const Problem& problem) {
    std::vector<SearchState> stack;
    stack.push_back({0, problem.initial_choices(), 0});
    while (!stack.empty()) {
        auto state = stack.back();
        stack.pop_back();
        if (state.position == problem.size()) {
            // Found a solution
            report(state.decisions);
            continue;
        }
        // Generate children – equivalent to recursive calls
        for (int choice = 0; choice < state.choices_remaining; ++choice) {
            if (problem.is_valid(state.position, choice)) {
                stack.push_back({
                    state.position + 1,

```



```

        problem.choices_at(state.position + 1),
        state.decisions | (choice << (state.position * 2))
    });
}
}
}
}
}

```

More code? Yes. Faster? Absolutely. Debuggable? Much more so — you can inspect the entire stack as a flat data structure instead of unwinding frames in GDB.

4. Tail Recursion Elimination

Tail recursion is when the recursive call is the **very last thing** the function does — no computation happens after it returns. In theory, the compiler can optimize this into a jump, reusing the current stack frame instead of allocating a new one.

```

// Tail-recursive factorial – the recursive call is the final operation
uint64_t factorial_tail(int n, uint64_t acc = 1) {
    if (n <= 1) return acc;
    return factorial_tail(n - 1, acc * n); // nothing happens after this return
}

```

```

// NOT tail-recursive – multiplication happens AFTER the recursive call returns
uint64_t factorial_not_tail(int n) {
    if (n <= 1) return 1;
    return n * factorial_not_tail(n - 1); // must multiply after call returns
}

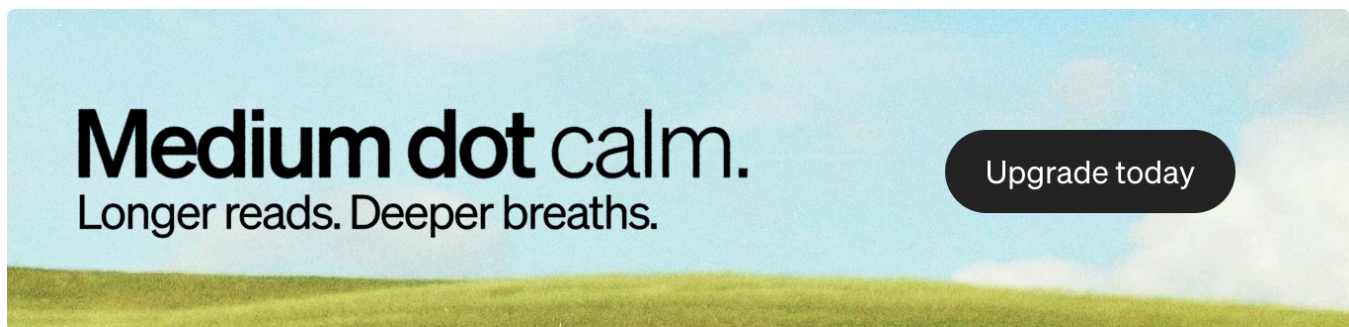
```

The Problem: C++ Doesn't Guarantee TCO

Unlike Scheme or Haskell, the C++ standard says absolutely nothing about tail call optimization. Whether it happens depends on:

FACTOR	TCO APPLIED?
GCC/Clang at <code>-O2</code> or higher, simple function	Usually yes
Virtual function	No
Non-trivial destructor on return type	No
<code>-fsanitize=address</code> enabled	No
Debug build (<code>-O0</code>)	Never
MSVC	Rarely

This means your “tail-recursive” function might be perfectly optimized on your machine, silently stack-overflow in CI with ASAN enabled, and explode in production on a different compiler. That’s not a trade-off. That’s a landmine.



How to Check:

```
g++ -O2 -S factorial.cpp
grep -E 'call|jmp' factorial.s | grep factorial
```

If you see `jmp factorial_tail` → TCO applied, the recursive call became a loop. If you see `call factorial_tail` → TCO did not apply. You're allocating stack frames.

My Advice:

Don't rely on it. If you care enough to write tail-recursive code, you care enough to

just write the loop:

```
// Just write the loop. Three lines. No ambiguity. No compiler prayers.
uint64_t factorial_iter(int n) noexcept {
    uint64_t acc = 1;
    for (int i = 2; i <= n; ++i) acc *= i;
    return acc;
}
```

5. Raise the Base Case Threshold

Instead of recursing all the way down to the trivial base case ($n=1$ or $n=0$), **stop earlier** and handle small inputs with a non-recursive algorithm. This is like adding an express elevator that skips the bottom 30 floors — you still get there, but with dramatically fewer stops.

```
// Standard mergesort – recurses all the way to single elements
void mergesort(int* data, int n) {
    if (n <= 1) return; // base case: 1 element
    int mid = n / 2;
    mergesort(data, mid);
    mergesort(data + mid, n - mid);
    merge(data, mid, n);
}
```

The problem: for $n=1024$, this generates 2047 recursive calls — a full binary tree of function invocations just to sort 1024 integers.

```
// Higher base level – switch to insertion sort at 32 elements
void mergesort_fast(int* data, int n) {
    if (n <= 32) {
        // Insertion sort: no recursion, fits in L1 cache,
        // branch predictor loves the nearly-sorted inner loop.
        insertion_sort(data, n);
        return;
    }
}
```

```

    int mid = n / 2;
    mergesort_fast(data, mid);
    mergesort_fast(data + mid, n - mid);
    merge(data, mid, n);
}

// Simple, cache-friendly, perfect for small arrays
void insertion_sort(int* data, int n) noexcept {
    for (int i = 1; i < n; ++i) {
        int key = data[i];
        int j = i - 1;
        while (j >= 0 && data[j] > key) {
            data[j + 1] = data[j];
            --j;
        }
        data[j + 1] = key;
    }
}

```

The numbers:

ARRAY SIZE	BASE=1 CALLS	BASE=32 CALLS	REDUCTION
1,024	2,047	63	97%
65,536	131,071	4,095	97%
1,048,576	2,097,151	65,535	97%

97% fewer function calls. And the insertion sort on 32 elements is essentially free — the entire working set fits in two cache lines.

This is exactly what `std::sort` does. The standard library uses introsort (quicksort + heapsort fallback) with insertion sort for small partitions. They chose a threshold around 16-32 elements. The library authors figured this out decades ago. Steal from the best.

Other places this applies:

- **Quicksort** → switch to insertion sort below threshold
- **Recursive FFT** → switch to a hardcoded butterfly at size 4 or 8

- **Recursive matrix multiply** → switch to a tiled iterative kernel below 64×64

6. Collapse Recursion Levels

Instead of recursing one level at a time, process multiple levels' worth of work in a single call. This is the recursion equivalent of loop unrolling — you do more useful work per function invocation, reducing the ratio of overhead to actual computation.

Standard: One Node Per Call

```
int tree_sum(const TreeNode* node) {  
    if (!node) return 0;  
    return node->value + tree_sum(node->left) + tree_sum(node->right);  
}
```

Every node = one function call. For a tree with 1 million nodes, that's 1 million calls.

Collapsed: Two Levels Per Call

```
int tree_sum_collapsed(const TreeNode* node) {  
    if (!node) return 0;  
    int sum = node->value;  
    // Process left child AND its children - two levels in one frame  
    if (node->left) {  
        sum += node->left->value;  
        sum += tree_sum_collapsed(node->left->left);  
        sum += tree_sum_collapsed(node->left->right);  
    }  
    // Same for right child  
    if (node->right) {  
        sum += node->right->value;  
        sum += tree_sum_collapsed(node->right->left);  
        sum += tree_sum_collapsed(node->right->right);  
    }  
    return sum;  
}
```

The math:

TREE DEPTH	STANDARD CALLS	COLLAPSED (2-LEVEL)	REDUCTION
10 (~1K nodes)	1,023	~341	67%
20 (~1M nodes)	1,048,575	~349,525	67%
30 (~1B nodes)	~1 billion	~333 million	67%

Each call does 3x the useful work (processes 3 nodes instead of 1), so you need roughly 1/3 the calls. You can collapse 3 or 4 levels for even more savings, but the code gets ugly fast.

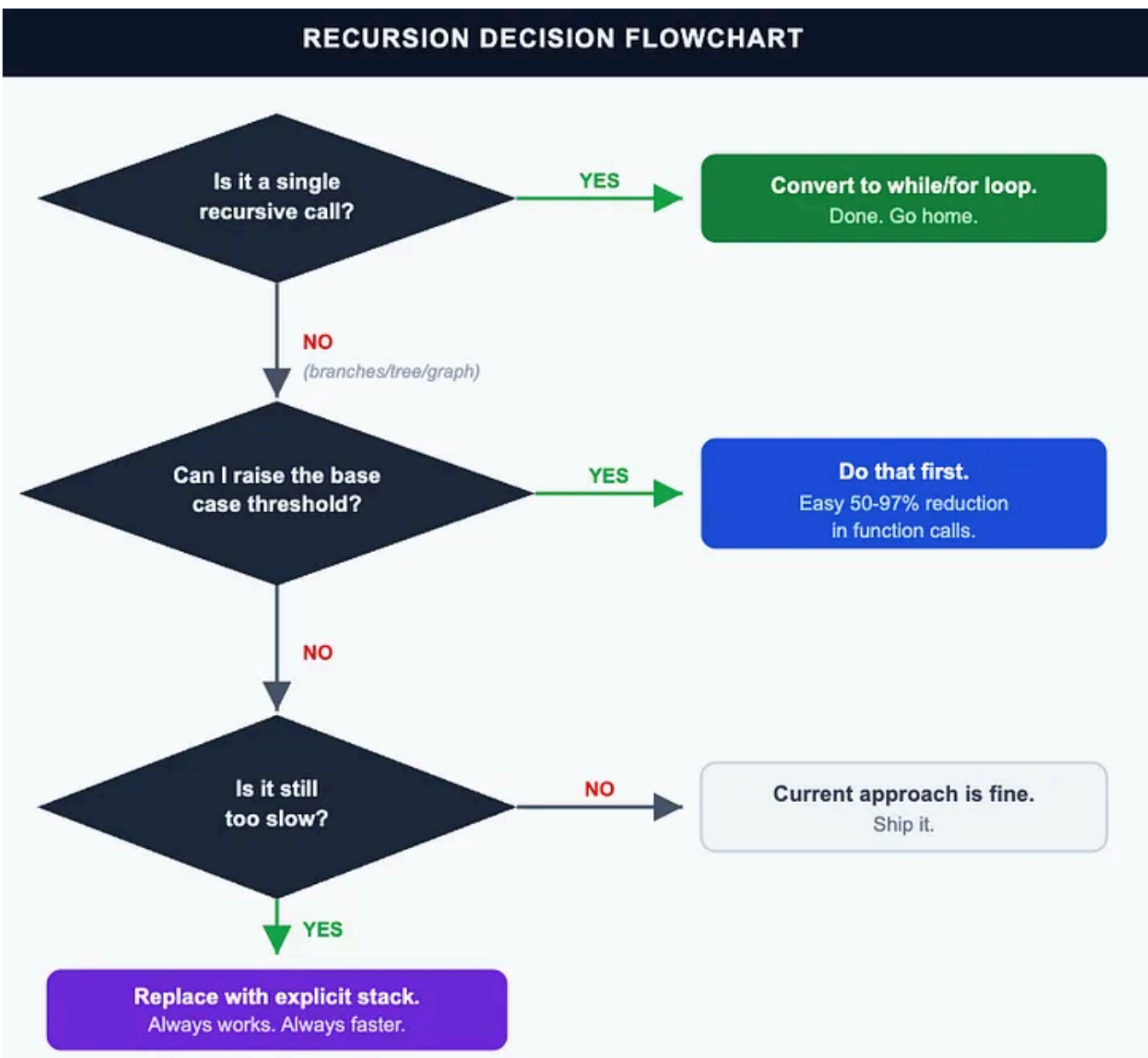
When to Use This vs. Explicit Stack

SITUATION	RECOMMENDATION
Hot path, maximum performance	Explicit stack (Section 3)
Moderate performance need, want readable code	Collapse 2 levels
Cold path, clarity matters most	Standard recursion is fine

Honest take: If you’re collapsing recursion levels, you’ve already acknowledged that the recursion overhead is a problem. At that point, just go all the way and use an explicit stack. It’s cleaner, faster, and doesn’t make the next developer’s eyes bleed parsing triple-nested null checks.

Quick Reference

OPTIMIZATION	WHAT IT DOES	IMPACT
Avoid recursion entirely	Eliminate all call overhead	Highest
Simple recursion → loop	Single-call recursion becomes iteration	High
Complex recursion → explicit stack	Branch recursion without call frames	High
Tail recursion elimination	Compiler reuses stack frame (maybe)	Medium
Higher base level	Fewer calls, better cache behavior	High
Collapse recursion levels	Multiple levels of work per call	Medium-High



Final Thoughts

Some key points to remember:

1. **Recursion is for whiteboards, not production.** It's a great way to *think* about problems. It's a terrible way to *solve* them in performance-critical code. Sketch the recursive solution on paper, then implement the iterative version.
2. **The explicit stack pattern is universal.** Any recursive algorithm can be converted. It's mechanical. It's always faster. Learn the pattern once, apply it forever.
3. **Profile before and after.** Use `perf stat -e cycles,instructions,branch-misses,cache-misses taskset -c 0 ./your_binary` to see the difference. The numbers are always more dramatic than you expect.
4. **`noexcept` your iterative replacements.** They can't throw (no stack overflow, no recursive depth issues), so mark them `noexcept` and let the compiler optimize accordingly.
5. **Check your standard library.** `std::sort` doesn't recurse to size 1. `std::stable_sort` uses insertion sort for small ranges. The library authors already made these decisions. If your custom algorithm recurses to the base case, you're leaving performance on the table.

Remember Bill Murray. He didn't fix Groundhog Day by waking up faster each morning. He fixed it by breaking out of the loop entirely. Do the same with your recursion.

Found this article helpful? Please **Clap** 🙌 and **follow** for more C++ and system programming content.

Programming

Software Development

C Plus Plus Language

Cpp

Performance